МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №8

по дисциплине «Вычислительная математика»

Тема: Решение систем линейных уравнений методом Якоби.

Студент гр. 4342 Тиссен М.Ю.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Реализовать метод Якоби для решения систем линейных алгебраических уравнений, исследовать его работу на матрицах различных типов и размеров, проверить точность получаемого решения по сравнению с заранее известным точным решением и по величине невязки, а также проанализировать влияние диагонального преобладания на сходимость метода.

Теоретический минимум.

Метод Якоби относится к итерационным методам решения систем линейных уравнений. Его идея состоит в том, что на каждой итерации новое приближение к решению вычисляется по значениям предыдущей итерации. В отличие от прямых методов, здесь решение находится не за конечное число шагов исключения, а последовательным уточнением приближений.

Для системы Ax=b каждая компонента нового приближения вычисляется по формуле, в которой диагональный элемент строки используется в знаменателе, а все остальные слагаемые берутся из предыдущего приближения. Поэтому важным условием применимости метода является отсутствие нулей на диагонали матрицы.

Сходимость метода Якоби в значительной степени зависит от свойств матрицы. Наиболее надежно метод работает для матриц с диагональным преобладанием, когда модуль диагонального элемента в каждой строке больше или не меньше суммы модулей остальных элементов этой строки. Именно поэтому в данной работе основной упор сделан на диагонально доминирующие матрицы. Для матриц без диагонального преобладания сходимость не гарантируется: метод может сходиться медленно, не достигать нужной точности за заданное число итераций или вовсе расходиться.

Точность итерационного метода обычно оценивается не только по найденному вектору решения, но и по невязке Ax-b. В работе используются нормы L1, L2, Linf, которые позволяют оценить суммарную ошибку, общий размер отклонения и максимальную ошибку по одной компоненте соответственно.

Выполнение работы.

В работе реализован метод Якоби с начальным приближением, равным нулевому вектору. На каждой итерации вычисляется новое приближение решения, после чего определяется величина шага $||x^{(k+1)} - x^{(k)}||\infty$.Если эта величина становится меньше заданной точности, процесс считается сошедшимся. Если на диагонали матрицы встречается нулевой элемент, метод не применяется. Также в программе предусмотрены случаи достижения лимита итераций и расходимости.

Основной набор экспериментов проводится на случайных диагонально доминирующих матрицах разных размеров, так как именно для них метод Якоби наиболее естественен и устойчив. Дополнительно рассматриваются и другие типы матриц: случайная плотная матрица без специальных свойств, матрицы Гильберта, случайные почти вырожденные матрицы, вырожденная матрица и матрица с нулевой диагональю. Это позволяет сравнить поведение метода в благоприятных и неблагоприятных условиях и показать, что отсутствие диагонального преобладания ухудшает или вообще нарушает сходимость.

Для тестов, где нужно проверить точность решения, сначала задается точный вектор x*, затем по формуле b=Ax* строится правая часть системы. После этого методом Якоби находится приближенное решение x, и оно сравнивается с точным. Дополнительно вычисляется невязка, а также нормы ошибки решения и нормы невязки L1, L2, Linf.

В программе также анализируется диагональное преобладание матрицы: проверяется, является ли оно строгим, нестрогим или отсутствует, а также вычисляется отношение суммы внедиагональных элементов строки к модулю диагонального элемента. Это позволяет связать экспериментальные результаты со свойствами матрицы и объяснить, почему в одних случаях метод сходится хорошо, а в других работает хуже.

Для больших матриц полная матрица не выводится в консоль. Если размер не слишком велик, система сохраняется в отдельный текстовый файл, чтобы при необходимости ее можно было показать отдельно. Для каждого эксперимента также фиксируются число итераций, время работы и статус завершения метода. Итоговые результаты сохраняются в CSV-файл.

Выводы.

В ходе работы был исследован метод Якоби для решения систем линейных алгебраических уравнений. Проведенные эксперименты показали, что наиболее надежно метод работает на диагонально доминирующих матрицах, для которых обычно наблюдается устойчивая сходимость и приемлемая точность. Для матриц без выраженного диагонального преобладания, а также для плохо обусловленных, вырожденных или содержащих нули на диагонали систем, работа метода заметно ухудшается: сходимость замедляется, может не достигаться нужная точность или метод становится неприменимым. Таким образом, результаты подтверждают, что для метода Якоби свойства матрицы играют решающую роль, а диагональное преобладание является одним из главных условий его успешного применения.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.cpp

```
#include <clocale>
#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdint>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <limits>
#include <random>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

#include "methods.h"

#ifdef _WIN32
#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>
#endif

namespace {

constexpr double kZeroTolerance = 1e-12;
constexpr double kJacobiEps = 1e-10;
constexpr std::size_t kJacobiMaxIterations = 10000;
constexpr std::size_t kVeryLargeN = 10000;
constexpr std::size_t kFullMatrixLimit = 10;
constexpr std::size_t kPreviewVectorItems = 8;
constexpr std::size_t kMaxDumpMatrixN = 1000;

struct ExperimentReport {
    std::string name;
    std::string matrixType;
    std::string note;
    std::string systemPreview;
    std::string systemDumpFile;
    std::uint64_t seed = 0;
    std::size_t n = 0;
    bool executed = false;
    JacobiStatus status = JacobiStatus::MaxIterationsReached;
    std::size_t iterations = 0;
    double elapsedMs = 0.0;
    double lastStepLinf = std::numeric_limits<double>::quiet_NaN();
    DominanceInfo dominance{};
    VectorNorms solutionError{
        std::numeric_limits<double>::quiet_NaN(),
        std::numeric_limits<double>::quiet_NaN(),
        std::numeric_limits<double>::quiet_NaN()
    };
    VectorNorms residual{
```

```
        std::numeric_limits<double>::quiet_NaN(),
        std::numeric_limits<double>::quiet_NaN(),
        std::numeric_limits<double>::quiet_NaN()
    };
};

void SetupConsole() {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
#endif
    std::setlocale(LC_ALL, ".UTF-8");
}

bool HasFlag(int argc, char* argv[], const std::string& flag) {
    for (int index = 1; index < argc; ++index) {
        if (flag == argv[index]) {
            return true;
        }
    }
    return false;
}

std::uint64_t GetRunSeed(int argc, char* argv[]) {
    const std::string prefix = "--seed=";
    for (int index = 1; index < argc; ++index) {
        const std::string argument = argv[index];
        if (argument.rfind(prefix, 0) == 0) {
            return std::stoull(argument.substr(prefix.size()));
        }
        if (argument == "--seed" && index + 1 < argc) {
            return std::stoull(argv[index + 1]);
        }
    }

    std::random_device device;
    const std::uint64_t high = static_cast<std::uint64_t>(device()) << 32;
    const std::uint64_t low = static_cast<std::uint64_t>(device());
    return high ^ low;
}

std::uint64_t DeriveSeed(std::uint64_t seed, std::uint64_t salt) {
    return seed + 0x9E3779B97F4A7C15ULL * salt;
}

std::string SanitizeFileName(const std::string& text) {
    std::string result;
    result.reserve(text.size());

    for (char ch : text) {
        const unsigned char c = static_cast<unsigned char>(ch);
        if ((c >= 'a' && c <= 'z') ||
            (c >= 'A' && c <= 'Z') ||
            (c >= '0' && c <= '9')) {
            result.push_back(static_cast<char>(std::tolower(c)));
        } else {
            result.push_back('_');
        }
    }

    while (result.find("__") != std::string::npos) {
```

```
        result.erase(result.find("__"), 1);
    }
    while (!result.empty() && result.front() == '_') {
        result.erase(result.begin());
    }
    while (!result.empty() && result.back() == '_') {
        result.pop_back();
    }

    if (result.empty()) {
        return "system";
    }
    return result;
}

std::string FormatVectorPreview(const std::vector<double>& values,
                                const std::string& title) {
    const bool fullVector = values.size() <= kFullMatrixLimit;
    std::ostringstream out;
    out << std::fixed << std::setprecision(6);

    if (fullVector) {
        out << title << " = [";
        for (std::size_t index = 0; index < values.size(); ++index) {
            out << values[index];
            if (index + 1 < values.size()) {
                out << ", ";
            }
        }
        out << "]\n";
        return out.str();
    }

    const std::size_t shown = std::min(values.size(), kPreviewVectorItems);
    out << title << ", первые " << shown
        << " элементов из " << values.size() << ": [";
    for (std::size_t index = 0; index < shown; ++index) {
        out << values[index];
        if (index + 1 < shown) {
            out << ", ";
        }
    }
    out << "]\n";
    return out.str();
}

void WriteSystemDumpFile(const std::string& filename,
                         const DenseMatrix& matrix,
                         const std::vector<double>& b,
                         const std::vector<double>* exactX) {
    std::ofstream out(filename);
    out << std::fixed << std::setprecision(12);
     out << "Размер матрицы: " << matrix.rows() << "x" << matrix.cols() <<
"\n";
    out << "Матрица A:\n";
    for (std::size_t row = 0; row < matrix.rows(); ++row) {
        for (std::size_t col = 0; col < matrix.cols(); ++col) {
            out << matrix(row, col);
            if (col + 1 < matrix.cols()) {
                out << ' ';
            }
```

```
        }
        out << '\n';
    }

    out << "\nВектор b:\n";
    for (double value : b) {
        out << value << '\n';
    }

    if (exactX != nullptr) {
        out << "\nТочное решение x*:\n";
        for (double value : *exactX) {
            out << value << '\n';
        }
    }
}

std::string BuildSystemPreview(const std::string& experimentName,
                               std::uint64_t seed,
                               const DenseMatrix& matrix,
                               const std::vector<double>& b,
                               const std::vector<double>* exactX,
                               std::string& dumpFile) {
    std::ostringstream out;

          if   (matrix.rows()   <=   kFullMatrixLimit   &&   matrix.cols()   <=
kFullMatrixLimit) {
        out << std::fixed << std::setprecision(6);
        out << "Матрица A:\n";
        for (std::size_t row = 0; row < matrix.rows(); ++row) {
            out << "  [";
            for (std::size_t col = 0; col < matrix.cols(); ++col) {
                out << std::setw(12) << matrix(row, col);
                if (col + 1 < matrix.cols()) {
                    out << ' ';
                }
            }
            out << " ]\n";
        }
        out << FormatVectorPreview(b, "Вектор b");
        if (exactX != nullptr) {
            out << FormatVectorPreview(*exactX, "Точное решение x*");
        }
        return out.str();
    }

    if (matrix.rows() <= kMaxDumpMatrixN && matrix.cols() <= kMaxDumpMatrixN)
{
        dumpFile = "system_dump_" + SanitizeFileName(experimentName) +
                   "_n_" + std::to_string(matrix.rows()) +
                   "_seed_" + std::to_string(seed) + ".txt";
        WriteSystemDumpFile(dumpFile, matrix, b, exactX);
         out << "Матрица A не выводится в консоль. Полная система сохранена в
файл "
            << dumpFile << "\n";
    } else {
         out << "Матрица A не выводится в консоль. Размер слишком большой для
автоматической текстовой выгрузки ("
            << matrix.rows() << "x" << matrix.cols() << ").\n";
    }
```

```
    out << FormatVectorPreview(b, "Вектор b");
    if (exactX != nullptr) {
        out << FormatVectorPreview(*exactX, "Точное решение x*");
    }
    return out.str();
}

ExperimentReport RunExperiment(const std::string& name,
                               const std::string& matrixType,
                               const DenseMatrix& matrix,
                               const std::vector<double>& b,
                               const std::vector<double>* exactX,
                               const std::string& note,
                               std::uint64_t seed) {
    const auto start = std::chrono::steady_clock::now();
    const JacobiResult result = SolveJacobi(
        matrix,
        b,
        kJacobiEps,
        kJacobiMaxIterations,
        kZeroTolerance
    );
    const auto finish = std::chrono::steady_clock::now();

    ExperimentReport report;
    report.name = name;
    report.matrixType = matrixType;
    report.note = note;
     report.systemPreview = BuildSystemPreview(name, seed, matrix, b, exactX,
report.systemDumpFile);
    report.seed = seed;
    report.n = matrix.rows();
    report.executed = true;
    report.status = result.status;
    report.iterations = result.iterations;
    report.elapsedMs =
        std::chrono::duration<double, std::milli>(finish - start).count();
    report.lastStepLinf = result.lastStepLinf;
    report.dominance = AnalyzeDiagonalDominance(matrix, kZeroTolerance);

    if (!result.x.empty()) {
        report.residual =
            ComputeNorms(Subtract(Multiply(matrix, result.x), b));
    }

    if (exactX != nullptr && result.status == JacobiStatus::Converged) {
        report.solutionError = ComputeNorms(Subtract(result.x, *exactX));
    }

    return report;
}

ExperimentReport RunExperimentWithExactSolution(const std::string& name,
                                                const std::string&
matrixType,
                                                const DenseMatrix& matrix,
                                                   const std::vector<double>&
exactX,
                                                const std::string& note,
                                                std::uint64_t seed) {
    const std::vector<double> b = Multiply(matrix, exactX);
```

```
    return RunExperiment(name, matrixType, matrix, b, &exactX, note, seed);
}

ExperimentReport MakeSkippedLargeReport(std::uint64_t seed) {
    ExperimentReport report;
    report.name = "Случайная диагонально доминирующая матрица 10000x10000";
    report.matrixType = "случайная диагонально доминирующая";
    report.note =
          "По умолчанию тест пропущен. Для плотной матрицы 10000x10000 метод
Якоби слишком затратен по памяти и времени. Запустите программу с флагом
--run-10000 только при необходимости.";
    report.seed = seed;
    report.n = kVeryLargeN;
    report.executed = false;
    return report;
}

void PrintNormLine(const std::string& label, const VectorNorms& norms) {
    std::cout << "  " << label
              << ": L1 = " << std::scientific << norms.l1
              << ", L2 = " << norms.l2
              << ", Linf = " << norms.linf << '\n';
}

void PrintDominance(const DominanceInfo& dominance) {
    std::cout << "  Диагональное преобладание: ";
    if (dominance.zeroDiagonal) {
        std::cout << "есть нули на диагонали" << '\n';
        return;
    }

    if (dominance.strict) {
        std::cout << "строгое";
    } else if (dominance.weak) {
        std::cout << "нестрогое";
    } else {
        std::cout << "отсутствует";
    }

    std::cout << ", max(sum_off/|a_ii|) = "
              << std::scientific << dominance.maxRatio << '\n';
}

void PrintReport(const ExperimentReport& report) {
    std::cout << "Случай: " << report.name << '\n';
    std::cout << "  Размер n = " << report.n
              << ", тип матрицы = " << report.matrixType << '\n';
    if (report.seed != 0) {
        std::cout << "  Seed = " << report.seed << '\n';
    }
    if (!report.systemPreview.empty()) {
        std::cout << report.systemPreview;
    }
    if (!report.systemDumpFile.empty()) {
         std::cout << "  Файл с полной системой = " << report.systemDumpFile
<< '\n';
    }

    if (!report.executed) {
        std::cout << "  Статус = пропущен\n";
        std::cout << "  Оценка памяти под A, МиБ = "
```

```
                  << std::fixed << std::setprecision(3)
                  << EstimateDenseMatrixMiB(report.n) << '\n';
        std::cout << "  Оценка операций на одну итерацию = "
                                                  <<   std::scientific   <<
EstimateJacobiIterationFlops(report.n)
                  << '\n';
        std::cout << "  Примечание = " << report.note << "\n\n";
        return;
    }

    std::cout << "  Статус = " << ToString(report.status)
              << ", итераций = " << report.iterations
              << ", время, мс = " << std::fixed << std::setprecision(3)
              << report.elapsedMs << '\n';
    std::cout << "  Последний шаг ||x^(k+1)-x^k||_inf = "
              << std::scientific << report.lastStepLinf << '\n';
    PrintDominance(report.dominance);
    PrintNormLine("невязка", report.residual);

    if (!std::isnan(report.solutionError.l2)) {
        PrintNormLine("ошибка решения", report.solutionError);
    }

    if (!report.note.empty()) {
        std::cout << "  Примечание = " << report.note << '\n';
    }
    std::cout << '\n';
}

void WriteCsv(const std::string& filename,
              const std::vector<ExperimentReport>& reports) {
    std::ofstream out(filename);
    out << "случай;seed;тип_матрицы;n;выполнен;статус;итерации;время_мс;"
           "strict_dominance;weak_dominance;max_ratio;шаг_linf;файл_системы;"

"ошибка_l1;ошибка_l2;ошибка_linf;невязка_l1;невязка_l2;невязка_linf;примечани
e\n";
    out << std::setprecision(17);

    for (const auto& report : reports) {
        out << report.name << ';'
            << report.seed << ';'
            << report.matrixType << ';'
            << report.n << ';'
            << (report.executed ? 1 : 0) << ';'
              << (report.executed ? ToString(report.status) : "пропущен") <<
';'
            << report.iterations << ';'
            << report.elapsedMs << ';'
            << (report.dominance.strict ? 1 : 0) << ';'
            << (report.dominance.weak ? 1 : 0) << ';'
            << report.dominance.maxRatio << ';'
            << report.lastStepLinf << ';'
            << report.systemDumpFile << ';'
            << report.solutionError.l1 << ';'
            << report.solutionError.l2 << ';'
            << report.solutionError.linf << ';'
            << report.residual.l1 << ';'
            << report.residual.l2 << ';'
            << report.residual.linf << ';'
            << report.note << '\n';
```

```
    }
}

}  // namespace

int main(int argc, char* argv[]) {
    try {
        SetupConsole();
        const bool runVeryLarge = HasFlag(argc, argv, "--run-10000");
        const std::uint64_t runSeed = GetRunSeed(argc, argv);
        std::mt19937_64 seedGenerator(runSeed);
        const auto nextExperimentSeed = [&seedGenerator]() {
            return seedGenerator();
        };

        std::vector<ExperimentReport> reports;

        std::cout << std::fixed << std::setprecision(6);
        std::cout << "Решение СЛАУ методом Якоби\n";
         std::cout << "Порог нуля: " << std::scientific << kZeroTolerance <<
'\n';
        std::cout << "Точность eps: " << kJacobiEps << '\n';
        std::cout << "Лимит итераций: " << std::fixed << kJacobiMaxIterations
<< '\n';
        std::cout << "Базовый seed запуска: " << runSeed << "\n\n";

        {
            const DenseMatrix matrix = MakeMatrix(
                3, 3,
                {
                    10.0, -1.0, 2.0,
                    -1.0, 11.0, -1.0,
                    2.0, -1.0, 10.0
                }
            );
            const std::vector<double> xExact{1.0, 2.0, -1.0};
            reports.push_back(RunExperimentWithExactSolution(
                "Базовый пример 3x3",
                "заданная вручную диагонально доминирующая",
                matrix,
                xExact,
                "Классический небольшой пример, на котором метод Якоби должен
сходиться быстро.",
                0
            ));
        }

           for (const std::size_t n : std::vector<std::size_t>{10, 30, 100,
300}) {
            const std::uint64_t seed = nextExperimentSeed();
            const DenseMatrix matrix = MakeRandomDiagonalDominantMatrix(
                n,
                DeriveSeed(seed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                n,
                DeriveSeed(seed, 2)
            );
            reports.push_back(RunExperimentWithExactSolution(
                        "Случайная диагонально доминирующая матрица n=" +
std::to_string(n),
```

```
                "случайная диагонально доминирующая",
                matrix,
                xExact,
                "Основной набор тестов для метода Якоби.",
                seed
            ));
        }

        {
            const std::uint64_t seed = nextExperimentSeed();
            const DenseMatrix matrix = MakeRandomDenseMatrix(
                12,
                DeriveSeed(seed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                12,
                DeriveSeed(seed, 2)
            );
            reports.push_back(RunExperimentWithExactSolution(
                "Случайная плотная матрица 12x12",
                "случайная плотная",
                matrix,
                xExact,
                  "Для случайной плотной матрицы сходимость не гарантируется.
Тест нужен для сравнения с диагонально доминирующими случаями.",
                seed
            ));
        }

        for (const std::size_t n : std::vector<std::size_t>{5, 8}) {
            const std::uint64_t seed = nextExperimentSeed();
            const DenseMatrix matrix = MakeHilbertMatrix(n);
            const std::vector<double> xExact(n, 1.0);
            reports.push_back(RunExperimentWithExactSolution(
                "Матрица Гильберта n=" + std::to_string(n),
                "матрица Гильберта",
                matrix,
                xExact,
                   "Классический плохо обусловленный тест. Обычно для метода
Якоби ведет себя заметно хуже диагонально доминирующих матриц.",
                seed
            ));
        }

        for (const std::size_t n : std::vector<std::size_t>{8, 12}) {
            const std::uint64_t seed = nextExperimentSeed();
            const DenseMatrix matrix = MakeRandomNearlySingularMatrix(
                n,
                DeriveSeed(seed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                n,
                DeriveSeed(seed, 2)
            );
            reports.push_back(RunExperimentWithExactSolution(
                "Случайная почти вырожденная матрица n=" + std::to_string(n),
                "случайная почти вырожденная",
                matrix,
                xExact,
                        "Матрица построена как почти линейно зависимая. Это
дополнительная проверка устойчивости метода.",
```

```
                seed
            ));
        }

        {
            const std::uint64_t seed = nextExperimentSeed();
            const DenseMatrix matrix = MakeRandomRankDeficientMatrix(
                8,
                DeriveSeed(seed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                8,
                DeriveSeed(seed, 2)
            );
            reports.push_back(RunExperimentWithExactSolution(
                "Случайная вырожденная матрица 8x8",
                "случайная вырожденная",
                matrix,
                xExact,
                      "Для вырожденных матриц метод Якоби не имеет гарантии
получения корректного ответа.",
                seed
            ));
        }

        {
            const std::uint64_t seed = nextExperimentSeed();
            const DenseMatrix matrix = MakeRandomZeroDiagonalMatrix(
                6,
                DeriveSeed(seed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                6,
                DeriveSeed(seed, 2)
            );
            reports.push_back(RunExperimentWithExactSolution(
                "Случайная матрица 6x6 с нулевой диагональю",
                "случайная с нулевой диагональю",
                matrix,
                xExact,
                      "Если на диагонали есть ноль, формулы Якоби применить
нельзя.",
                seed
            ));
        }

        {
            const std::uint64_t seed = nextExperimentSeed();
            const DenseMatrix matrix = MakeRandomDiagonalDominantMatrix(
                1000,
                DeriveSeed(seed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                1000,
                DeriveSeed(seed, 2)
            );
            reports.push_back(RunExperimentWithExactSolution(
                "Случайная диагонально доминирующая матрица 1000x1000",
                "случайная диагонально доминирующая",
                matrix,
                xExact,
```

```
                    "Большой устойчивый тест для оценки скорости сходимости и
времени работы.",
                seed
            ));
        }

        const std::uint64_t largeSeed = nextExperimentSeed();
        if (runVeryLarge) {
            const DenseMatrix matrix = MakeRandomDiagonalDominantMatrix(
                kVeryLargeN,
                DeriveSeed(largeSeed, 1)
            );
            const std::vector<double> xExact = MakeRandomVector(
                kVeryLargeN,
                DeriveSeed(largeSeed, 2)
            );
            reports.push_back(RunExperimentWithExactSolution(
                "Случайная диагонально доминирующая матрица 10000x10000",
                "случайная диагонально доминирующая",
                matrix,
                xExact,
                    "Эксперимент включен через флаг --run-10000. Для плотной
матрицы этот тест очень тяжелый.",
                largeSeed
            ));
        } else {
            reports.push_back(MakeSkippedLargeReport(largeSeed));
        }

        for (const auto& report : reports) {
            PrintReport(report);
        }

        WriteCsv("jacobi_results.csv", reports);
        std::cout << "CSV-отчет сохранен в файл jacobi_results.csv\n";
        return 0;
    } catch (const std::exception& error) {
        std::cerr << "Ошибка: " << error.what() << '\n';
        return 1;
    }
}
```

Файл methods.cpp

```
#include "methods.h"


#include <algorithm>

#include <cmath>

#include <limits>

#include <random>

#include <stdexcept>


DenseMatrix::DenseMatrix() : rows_(0), cols_(0) {

}
```

```
DenseMatrix::DenseMatrix(std::size_t rows, std::size_t cols)
    : rows_(rows),
      cols_(cols),
      data_(rows * cols, 0.0) {
}

std::size_t DenseMatrix::rows() const {
    return rows_;
}

std::size_t DenseMatrix::cols() const {
    return cols_;
}

double& DenseMatrix::operator()(std::size_t row, std::size_t col) {
    return data_[row * cols_ + col];
}

double DenseMatrix::operator()(std::size_t row, std::size_t col) const {
    return data_[row * cols_ + col];
}

JacobiResult SolveJacobi(const DenseMatrix& matrix,
                         const std::vector<double>& b,
                         double eps,
                         std::size_t maxIterations,
                         double zeroTolerance,
                         double divergenceThreshold) {
    if (matrix.rows() != matrix.cols()) {
        throw std::invalid_argument("Matrix must be square.");
    }
    if (b.size() != matrix.rows()) {
        throw std::invalid_argument("Vector size must match matrix size.");
    }

    const std::size_t n = matrix.rows();
    for (std::size_t i = 0; i < n; ++i) {
        if (std::abs(matrix(i, i)) <= zeroTolerance) {
            JacobiResult result;
            result.status = JacobiStatus::ZeroOnDiagonal;
```

```
            return result;
        }
    }

    std::vector<double> current(n, 0.0);
    std::vector<double> next(n, 0.0);

    for (std::size_t iteration = 1; iteration <= maxIterations; ++iteration)
{
        double stepLinf = 0.0;

        for (std::size_t row = 0; row < n; ++row) {
            long double sum = 0.0;
            for (std::size_t col = 0; col < n; ++col) {
                if (col == row) {
                    continue;
                }
                sum += static_cast<long double>(matrix(row, col)) *
current[col];
            }

            next[row] = static_cast<double>(
                (static_cast<long double>(b[row]) - sum) / matrix(row, row)
            );
            stepLinf = std::max(stepLinf, std::abs(next[row] -
current[row]));
        }

        if (!std::isfinite(stepLinf) || stepLinf > divergenceThreshold) {
            JacobiResult result;
            result.status = JacobiStatus::Diverged;
            result.x = std::move(next);
            result.iterations = iteration;
            result.lastStepLinf = stepLinf;
            return result;
        }

        current.swap(next);

        if (stepLinf <= eps) {
            JacobiResult result;
```

```
            result.status = JacobiStatus::Converged;
            result.x = std::move(current);
            result.iterations = iteration;
            result.lastStepLinf = stepLinf;
            return result;
        }
    }

    JacobiResult result;
    result.status = JacobiStatus::MaxIterationsReached;
    result.x = std::move(current);
    result.iterations = maxIterations;
    result.lastStepLinf = 0.0;
    return result;
}

DenseMatrix MakeMatrix(std::size_t rows,
                       std::size_t cols,
                       const std::vector<double>& values) {
    if (rows * cols != values.size()) {
        throw std::invalid_argument("Row-major value count is incorrect.");
    }

    DenseMatrix matrix(rows, cols);
    for (std::size_t row = 0; row < rows; ++row) {
        for (std::size_t col = 0; col < cols; ++col) {
            matrix(row, col) = values[row * cols + col];
        }
    }
    return matrix;
}

DenseMatrix MakeHilbertMatrix(std::size_t n) {
    DenseMatrix matrix(n, n);
    for (std::size_t row = 0; row < n; ++row) {
        for (std::size_t col = 0; col < n; ++col) {
            matrix(row, col) = 1.0 / static_cast<double>(row + col + 1);
        }
    }
    return matrix;
}
```

```
DenseMatrix MakeRandomDenseMatrix(std::size_t n,
                                  std::uint64_t seed,
                                  double minValue,
                                  double maxValue) {
    if (!(minValue < maxValue)) {
        throw std::invalid_argument("Random interval must be non-empty.");
    }

    std::mt19937_64 rng(seed);
    std::uniform_real_distribution<double> distribution(minValue, maxValue);

    DenseMatrix matrix(n, n);
    for (std::size_t row = 0; row < n; ++row) {
        for (std::size_t col = 0; col < n; ++col) {
            double value = distribution(rng);
            if (std::abs(value) < 0.1) {
                value = (value < 0.0) ? -0.1 : 0.1;
            }
            matrix(row, col) = value;
        }
    }
    return matrix;
}

DenseMatrix MakeRandomNearlySingularMatrix(std::size_t n,
                                           std::uint64_t seed,
                                           double perturbation) {
    if (perturbation <= 0.0) {
        throw std::invalid_argument("Perturbation must be positive.");
    }

    std::mt19937_64 rng(seed);
    std::uniform_real_distribution<double> valueDistribution(-2.0, 2.0);
    std::uniform_real_distribution<double> scaleDistribution(0.5, 2.0);
    std::uniform_real_distribution<double> noiseDistribution(-1.0, 1.0);

    std::vector<double> baseRow(n, 0.0);
    for (double& value : baseRow) {
        value = valueDistribution(rng);
        if (std::abs(value) < 0.2) {
```

```
            value = (value < 0.0) ? -0.2 : 0.2;
        }
    }

    DenseMatrix matrix(n, n);
    for (std::size_t row = 0; row < n; ++row) {
        const double scale = scaleDistribution(rng);
        for (std::size_t col = 0; col < n; ++col) {
            matrix(row, col) =
                scale * baseRow[col] + perturbation * noiseDistribution(rng);
        }
        matrix(row, row) += perturbation * static_cast<double>(row + 1);
    }
    return matrix;
}

DenseMatrix MakeRandomDiagonalDominantMatrix(std::size_t n,
                                             std::uint64_t seed,
                                             double minValue,
                                             double maxValue) {
    if (!(minValue < maxValue)) {
        throw std::invalid_argument("Random interval must be non-empty.");
    }

    std::mt19937_64 rng(seed);
    std::uniform_real_distribution<double> distribution(minValue, maxValue);
    std::uniform_real_distribution<double> diagonalGap(1.0, 3.0);

    DenseMatrix matrix(n, n);
    for (std::size_t row = 0; row < n; ++row) {
        double rowSum = 0.0;
        for (std::size_t col = 0; col < n; ++col) {
            if (row == col) {
                continue;
            }

            double value = distribution(rng);
            if (std::abs(value) < 0.05) {
                value = (value < 0.0) ? -0.05 : 0.05;
            }
```

```
            matrix(row, col) = value;
            rowSum += std::abs(value);
        }

        const double sign = (distribution(rng) >= 0.0) ? 1.0 : -1.0;
        matrix(row, row) = sign * (rowSum + diagonalGap(rng));
    }
    return matrix;
}

DenseMatrix MakeRandomRankDeficientMatrix(std::size_t n,
                                          std::uint64_t seed) {
    std::mt19937_64 rng(seed);
    std::uniform_real_distribution<double> valueDistribution(-3.0, 3.0);
    std::uniform_real_distribution<double> scaleDistribution(-2.0, 2.0);

    std::vector<double> baseRow(n, 0.0);
    for (double& value : baseRow) {
        value = valueDistribution(rng);
        if (std::abs(value) < 0.2) {
            value = (value < 0.0) ? -0.2 : 0.2;
        }
    }

    DenseMatrix matrix(n, n);
    for (std::size_t row = 0; row < n; ++row) {
        double scale = scaleDistribution(rng);
        if (std::abs(scale) < 0.3) {
            scale = (scale < 0.0) ? -1.0 : 1.0;
        }
        for (std::size_t col = 0; col < n; ++col) {
            matrix(row, col) = scale * baseRow[col];
        }
    }
    return matrix;
}

DenseMatrix MakeRandomZeroDiagonalMatrix(std::size_t n,
                                         std::uint64_t seed,
                                         double minValue,
                                         double maxValue) {
```

```
    DenseMatrix matrix = MakeRandomDenseMatrix(n, seed, minValue, maxValue);
    for (std::size_t i = 0; i < n; ++i) {
        matrix(i, i) = 0.0;
    }
    return matrix;
}

std::vector<double> MakeRandomVector(std::size_t n,
                                     std::uint64_t seed,
                                     double minValue,
                                     double maxValue) {
    if (!(minValue < maxValue)) {
        throw std::invalid_argument("Random interval must be non-empty.");
    }

    std::mt19937_64 rng(seed);
    std::uniform_real_distribution<double> distribution(minValue, maxValue);

    std::vector<double> values(n, 0.0);
    for (double& value : values) {
        value = distribution(rng);
    }
    return values;
}

std::vector<double> Multiply(const DenseMatrix& matrix,
                             const std::vector<double>& x) {
    if (matrix.cols() != x.size()) {
        throw std::invalid_argument("Matrix/vector dimensions do not
match.");
    }

    std::vector<double> result(matrix.rows(), 0.0);
    for (std::size_t row = 0; row < matrix.rows(); ++row) {
        long double sum = 0.0;
        for (std::size_t col = 0; col < matrix.cols(); ++col) {
            sum += static_cast<long double>(matrix(row, col)) * x[col];
        }
        result[row] = static_cast<double>(sum);
    }
    return result;
```

```
}

std::vector<double> Subtract(const std::vector<double>& left,
                             const std::vector<double>& right) {
    if (left.size() != right.size()) {
        throw std::invalid_argument("Vector dimensions do not match.");
    }

    std::vector<double> result(left.size(), 0.0);
    for (std::size_t index = 0; index < left.size(); ++index) {
        result[index] = left[index] - right[index];
    }
    return result;
}

VectorNorms ComputeNorms(const std::vector<double>& values) {
    long double l1 = 0.0;
    long double l2Squared = 0.0;
    double linf = 0.0;

    for (double value : values) {
        const double absolute = std::abs(value);
        l1 += absolute;
        l2Squared += static_cast<long double>(absolute) * absolute;
        linf = std::max(linf, absolute);
    }

    VectorNorms norms{};
    norms.l1 = static_cast<double>(l1);
    norms.l2 = std::sqrt(static_cast<double>(l2Squared));
    norms.linf = linf;
    return norms;
}

DominanceInfo AnalyzeDiagonalDominance(const DenseMatrix& matrix,
                                       double zeroTolerance) {
    if (matrix.rows() != matrix.cols()) {
        throw std::invalid_argument("Matrix must be square.");
    }

    DominanceInfo info;
```

```
    info.weak = true;
    info.strict = true;
    info.maxRatio = 0.0;

    for (std::size_t row = 0; row < matrix.rows(); ++row) {
        const double diagAbs = std::abs(matrix(row, row));
        if (diagAbs <= zeroTolerance) {
            info.zeroDiagonal = true;
            info.weak = false;
            info.strict = false;
            info.maxRatio = std::numeric_limits<double>::infinity();
            return info;
        }

        long double offDiagonalSum = 0.0;
        for (std::size_t col = 0; col < matrix.cols(); ++col) {
            if (col == row) {
                continue;
            }
            offDiagonalSum += std::abs(matrix(row, col));
        }

        const double ratio = static_cast<double>(offDiagonalSum / diagAbs);
        info.maxRatio = std::max(info.maxRatio, ratio);

        if (diagAbs + zeroTolerance < static_cast<double>(offDiagonalSum)) {
            info.weak = false;
            info.strict = false;
        } else if (diagAbs <= static_cast<double>(offDiagonalSum) +
zeroTolerance) {
            info.strict = false;
        }
    }

    return info;
}

double EstimateDenseMatrixMiB(std::size_t n) {
    const long double bytes =
        static_cast<long double>(n) * n * sizeof(double);
    return static_cast<double>(bytes / (1024.0L * 1024.0L));
```

```
}

long double EstimateJacobiIterationFlops(std::size_t n) {
    const long double size = static_cast<long double>(n);
    return 2.0L * size * size;
}

std::string ToString(JacobiStatus status) {
    switch (status) {
        case JacobiStatus::Converged:
            return "сошелся";
        case JacobiStatus::MaxIterationsReached:
            return "достигнут лимит итераций";
        case JacobiStatus::ZeroOnDiagonal:
            return "нулевой элемент на диагонали";
        case JacobiStatus::Diverged:
            return "расходится";
    }
    return "неизвестный статус";
}
```

Файл methods.h

```
#pragma once

#include <cstddef>
#include <cstdint>
#include <string>
#include <vector>

class DenseMatrix {
public:
    DenseMatrix();
    DenseMatrix(std::size_t rows, std::size_t cols);

    std::size_t rows() const;
    std::size_t cols() const;

    double& operator()(std::size_t row, std::size_t col);
```

```
    double operator()(std::size_t row, std::size_t col) const;

private:
    std::size_t rows_;
    std::size_t cols_;
    std::vector<double> data_;
};

enum class JacobiStatus {
    Converged,
    MaxIterationsReached,
    ZeroOnDiagonal,
    Diverged
};

struct VectorNorms {
    double l1;
    double l2;
    double linf;
};

struct DominanceInfo {
    bool weak = false;
    bool strict = false;
    bool zeroDiagonal = false;
    double maxRatio = 0.0;
};

struct JacobiResult {
    JacobiStatus status = JacobiStatus::MaxIterationsReached;
    std::vector<double> x;
    std::size_t iterations = 0;
    double lastStepLinf = 0.0;
};

JacobiResult SolveJacobi(const DenseMatrix& matrix,
                         const std::vector<double>& b,
                         double eps = 1e-10,
                         std::size_t maxIterations = 10000,
                         double zeroTolerance = 1e-12,
                         double divergenceThreshold = 1e100);
```

```
DenseMatrix MakeMatrix(std::size_t rows,
                       std::size_t cols,
                       const std::vector<double>& values);

DenseMatrix MakeHilbertMatrix(std::size_t n);
DenseMatrix MakeRandomDenseMatrix(std::size_t n,
                                  std::uint64_t seed,
                                  double minValue = -10.0,
                                  double maxValue = 10.0);
DenseMatrix MakeRandomNearlySingularMatrix(std::size_t n,
                                           std::uint64_t seed,
                                           double perturbation = 1e-6);
DenseMatrix MakeRandomDiagonalDominantMatrix(std::size_t n,
                                             std::uint64_t seed,
                                             double minValue = -3.0,
                                             double maxValue = 3.0);
DenseMatrix MakeRandomRankDeficientMatrix(std::size_t n,
                                          std::uint64_t seed);
DenseMatrix MakeRandomZeroDiagonalMatrix(std::size_t n,
                                         std::uint64_t seed,
                                         double minValue = -5.0,
                                         double maxValue = 5.0);

std::vector<double> MakeRandomVector(std::size_t n,
                                     std::uint64_t seed,
                                     double minValue = -5.0,
                                     double maxValue = 5.0);
std::vector<double> Multiply(const DenseMatrix& matrix,
                             const std::vector<double>& x);
std::vector<double> Subtract(const std::vector<double>& left,
                             const std::vector<double>& right);
VectorNorms ComputeNorms(const std::vector<double>& values);
DominanceInfo AnalyzeDiagonalDominance(const DenseMatrix& matrix,
                                       double zeroTolerance = 1e-12);

double EstimateDenseMatrixMiB(std::size_t n);
long double EstimateJacobiIterationFlops(std::size_t n);

std::string ToString(JacobiStatus status);
```